SELF-MAINTAINING ML SYSTEM

Full Project Documentation & Implementation Roadmap

Production monitoring • Drift detection • Delayed ground truth • Automated retraining • Champion/challenger promotion • Safe deployment

---

Primary use case: Customer churn prediction API

Intended environment: Local development on Windows with Docker Desktop, VS Code, Git and Python

Purpose: A self-contained specification that another AI assistant or developer can use to build the project in incremental, testable stages.

Version 1.0 • October 2026

How to use this document

This document defines the project goal, architecture, implementation sequence, technical choices, operational behavior, testing strategy, measurable evaluation and completion criteria. Treat it as the source of truth while implementing the project.

- Build one milestone at a time. Do not generate the entire codebase in one step.
- After each milestone, run the listed tests and commit the working result to Git.
- Keep the core project focused. Kubernetes, canary rollout and an LLM incident agent are optional extensions, not prerequisites.
- Use synthetic traffic and injected drift for controlled experiments, but label these experiments honestly. They demonstrate system behavior; they do not prove performance on a real production service.

Important engineering principle: input drift is evidence that the data distribution changed; it is not proof that predictive performance declined. Performance must be measured using ground-truth labels when they become available.

Contents

- 1. Project overview and objectives
- 2. Scope and design principles
- 3. Architecture and end-to-end workflow
- 4. Technology stack and environment
- 5. Repository structure
- 6. Data, labels and realistic simulation
- 7. Step-by-step implementation plan
- 8. Monitoring, drift detection and delayed labels
- 9. Automated retraining and model promotion
- 10. API, database and deployment behavior
- 11. Testing and failure scenarios
- 12. Experimental evaluation and metrics
- 13. Security, reliability and reproducibility
- 14. Final deliverables and acceptance checklist
- 15. Instructions for another AI coding assistant

1. Project overview and objectives

1.1 Project definition

Build a complete machine-learning operations (MLOps) system for a customer churn classifier. The model is served through an API. The system records requests and predictions, monitors service health and input distributions, evaluates performance when real labels arrive, retrains candidate models when justified, compares candidates with the deployed model, and promotes a candidate only if it passes explicit quality gates.

1.2 The problem

A model can lose effectiveness after deployment because customer behavior, product use, pricing, competition or data collection processes change. The deployed service may continue returning predictions while its assumptions become less reliable. Teams therefore need to monitor the service, identify distribution changes, measure performance when labels are available, and manage model updates safely.

1.3 Main objectives

- Serve reproducible churn predictions through FastAPI.
- Store prediction requests, predictions, timestamps, model versions and later-arriving labels.
- Collect service metrics such as latency, request rate, errors and prediction distribution.
- Compare recent input data with a fixed reference dataset using Evidently.
- Distinguish data drift from measured model performance degradation.
- Trigger or schedule a retraining workflow with data validation and model evaluation.
- Use MLflow to track experiments and model versions.
- Apply champion/challenger quality gates and reject unsafe candidates.
- Demonstrate recovery time, correct rejections, failure handling and reproducibility.

1.4 What success looks like

A reviewer can start the stack, send requests, inspect dashboards, inject a documented shift, observe a drift alert, simulate the arrival of ground-truth labels, run retraining, and see either a safe promotion or a deliberate rejection. Every important decision is logged and can be explained.

2. Scope and design principles

2.1 Core scope

Included in core	Not required for first release
Churn model and prediction API	Kubernetes and Helm
Request/prediction logging and label updates	GPU workloads
Drift and data-quality monitoring	LLM-based incident agent
Prometheus metrics, Grafana dashboards, alerts	Canary rollout / service mesh
Retraining workflow, MLflow registry	Multi-node or cloud infrastructure
Champion/challenger quality gates	Real customer PII
Docker Compose and CI tests	Unbounded automatic retraining

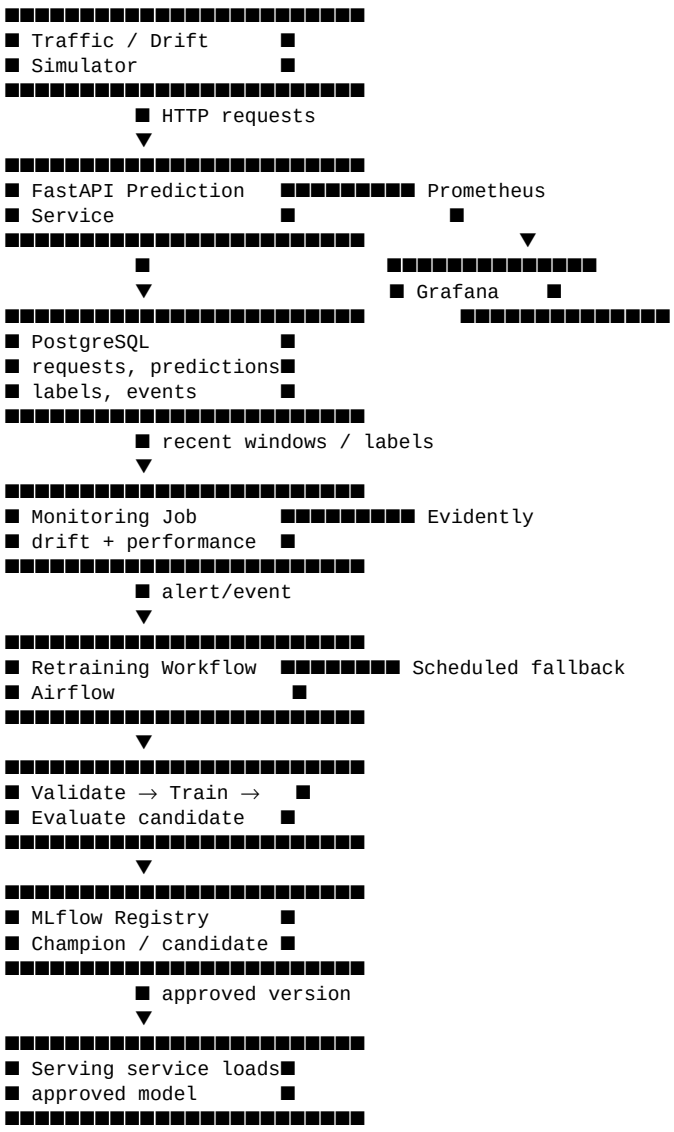
2.2 Design principles

- **Safety before automation:** a candidate never replaces the champion merely because drift was detected.

- **Labels matter:** do not pretend that unlabeled live predictions have known accuracy.
- **Reproducibility:** preserve data versions, code revision, parameters, metrics and model artifact.
- **Bounded automation:** use cooldowns, minimum sample sizes, retry limits and explicit quality gates.
- **Observable decisions:** record why retraining started, why a candidate passed or failed, and what model is active.
- **Incremental complexity:** first build a working end-to-end baseline, then add orchestration and dashboards.

3. Architecture and end-to-end workflow

3.1 Logical architecture



3.2 Service responsibilities

Component	Responsibility	Key boundary
FastAPI	Validate input, predict, log request and expose health/metrics endpoints	Should not train a model during a prediction request
PostgreSQL	Store prediction events, labels, monitoring results and retraining runs	Use stable IDs and timestamps
Monitoring job	Build windows, run drift and performance reports	Drift does not automatically equal failure
Prometheus	Scrape numeric service and monitoring metrics	Avoid high-cardinality labels such as customer IDs
Grafana	Visualize health, drift, performance and retraining status	Dashboards should distinguish unknown from poor performance
Alertmanager	Route threshold-based alerts to a webhook	Alert delivery must be idempotent
Airflow	Orchestrate validation, training, evaluation and promotion steps	Retries must not accidentally promote twice
MLflow	Track runs, artifacts and model versions	Registry metadata must include validation results

3.3 End-to-end workflow

- Train a baseline churn model and record baseline metrics.
- Register the model and mark the selected version as the initial champion.
- Start the API and load the approved model version.
- Replay traffic through the API; store features, prediction, model version and timestamp.
- Run monitoring over a minimum-sized recent window against a fixed reference dataset.
- If drift thresholds are crossed, create a monitoring event and alert. Do not assume performance has fallen.
- When true outcomes become available, attach labels to the original prediction records and calculate classification metrics.
- Trigger retraining only according to policy: drift plus sufficient evidence, measured degradation, or a scheduled check.
- Validate new training data, train a candidate, and evaluate it on a time-aware or otherwise appropriate holdout set.
- Compare candidate and champion under the same evaluation protocol and apply all quality gates.
- If approved, register/promote the candidate and make the serving service load that approved version. Otherwise retain the champion.
- Record outcomes and expose them in dashboards and an experiment report.

4. Technology stack and environment

Technology	Purpose	Implementation guidance
Python 3.11+	Application, training, monitoring and simulation	Use one documented version across local development and containers
pandas / NumPy	Tabular data handling	Use explicit schemas and stable feature names
scikit-learn; optional XGBoost	Baseline classifier and candidate training	Start with LogisticRegression or RandomForest; add complexity only if useful
FastAPI + Uvicorn	Prediction and operational endpoints	Separate API schemas from internal model features
PostgreSQL	Prediction log, delayed labels, run history	Use SQLAlchemy and Alembic migrations
MLflow	Experiment tracking, artifacts and model registry	Use a local tracking server and a persistent artifact location
Evidently	Data drift and data-quality reports	Pin a compatible version; its API can change between releases
Prometheus	Time-series metrics collection	Expose /metrics; define bounded metric labels
Grafana	Dashboards	Provision dashboards where practical
Alertmanager	Alert routing	Use webhook receiver with deduplication
Airflow	Workflow orchestration	Add after a manual retraining command works
Docker + Compose	Reproducible local services	Use health checks, named volumes and environment variables
GitHub Actions	CI: lint, tests, build checks	Do not deploy production automatically from an unreviewed model
pytest, Ruff	Testing and code quality	Run tests locally and in CI
Optuna (optional in core)	Hyperparameter optimization	Keep search budget small and reproducible
pandera (recommended)	Tabular data validation	Define required columns, types and value ranges

4.1 Local environment

Recommended for a student laptop: Windows 10/11, Docker Desktop with WSL 2, Git, VS Code and Python 3.11 or a compatible version. Airflow plus monitoring services can consume substantial memory. Start with the API, database and model first; add Airflow and the monitoring stack once the basic loop works.

- Minimum practical target: 8 GB RAM may work with a reduced stack; 16 GB is more comfortable for the full local stack.
- Use a Python virtual environment for local tests, but run services through Docker Compose where feasible.
- Do not commit credentials, `.env` files, customer data or large generated artifacts to Git.

4.2 Initial setup commands

```
# Create a project directory
mkdir self-maintaining-ml
cd self-maintaining-ml
git init

# Windows PowerShell: create a virtual environment
py -3.11 -m venv .venv
.\.venv\Scripts\Activate.ps1

# Install starter development dependencies
python -m pip install --upgrade pip
pip install pandas numpy scikit-learn fastapi uvicorn[standard]   sqlalchemy psycopg[binary] pydantic-settings pytest ru

# Save the initial environment snapshot
pip freeze > requirements-dev-lock.txt
```

For a team or longer-lived project, use a dependency manager or lock file and pin versions after verifying the full stack. Do not rely on an unpinned latest version for a reproducible final demonstration.

5. Repository structure

```
self-maintaining-ml/
├── app/
│   ├── api/
│   │   ├── main.py
│   │   ├── schemas.py
│   │   └── routes.py
│   ├── core/
│   │   ├── config.py
│   │   ├── logging.py
│   │   └── model_loader.py
│   ├── db/
│   │   ├── models.py
│   │   ├── session.py
│   │   └── migrations/
│   ├── monitoring/
│   │   ├── drift_report.py
│   │   ├── performance.py
│   │   └── metrics.py
│   └── training/
│       ├── data_validation.py
│       ├── train.py
│       ├── evaluate.py
│       └── promote.py
├── workflows/
│   ├── dags/
│       └── retraining_dag.py
├── data/
│   ├── raw/                # ignored by Git if data is large or sensitive
│   ├── processed/
│   └── reference/
├── simulator/
│   ├── replay.py
│   ├── inject_drift.py
│   └── deliver_labels.py
├── monitoring/
│   ├── prometheus.yml
│   ├── alert_rules.yml
│   ├── alertmanager.yml
│   └── grafana/
│       └── provisioning/
├── tests/
│   ├── unit/
│   ├── integration/
│   └── end_to_end/
├── scripts/
│   ├── initialize_db.py
│   ├── train_initial_model.py
│   └── run_experiment.py
├── docs/
│   ├── architecture.md
│   └── experiment-results.md
├── docker-compose.yml
├── Dockerfile
├── pyproject.toml
├── .env.example
├── .gitignore
└── README.md
```

The exact folders may evolve, but keep API, data access, training, monitoring, orchestration and simulation separate. This prevents monitoring or training code from becoming tangled with the prediction endpoint.

6. Data, labels and realistic simulation

6.1 Dataset

Start with the IBM Telco Customer Churn dataset, commonly available from public data repositories. Confirm the dataset's license and provenance before redistributing it. It contains customer/account features and a churn target. Keep a copy of the original source and document any cleaning decisions.

6.2 Data preparation

- Inspect column names, target values, missingness, duplicates and class balance.
- Define a target column such as `Churn` and document the positive class.
- Remove identifier-only fields such as customer ID from model features unless there is a justified use.
- Split data into train, validation and test sets before model selection; use a time-aware split if the dataset has a meaningful time dimension.
- Build preprocessing with a scikit-learn Pipeline/ColumnTransformer so training and serving use identical transformations.
- Keep a fixed reference dataset representing the baseline distribution.
- Store dataset version/hash, feature schema, split strategy, random seed and preprocessing configuration.

6.3 Avoid misleading drift simulations

A simple way to demonstrate data drift is to alter feature distributions in the simulator. However, merely changing age or income distributions does not guarantee that churn relationships change or that a model becomes less accurate. Treat feature drift and concept drift as different experimental conditions.

Scenario	What changes	What it demonstrates	What must not be claimed
Baseline	Same distribution as training/reference	Normal service and baseline metrics	Not proof of production readiness
Covariate/feature drift	One or more input feature distributions change	Detection of distribution shift	Does not prove accuracy fell
Label/prevalence shift	Churn prevalence changes	Prediction/label distribution monitoring	May require threshold and calibration analysis
Concept drift simulation	Relationship between features and target changes	Potential performance degradation	A synthetic transformation is not real customer behavior
Operational incident	Latency/errors rise or service fails	Service alerting and recovery	Not a model-quality issue by itself

6.4 Delayed ground truth

At prediction time, the service usually does not know whether a customer will actually churn. Store a unique prediction ID and later attach the observed outcome when it becomes available. The label-delivery simulator should reproduce this delay.

```
prediction_events
- prediction_id (UUID, primary key)
- event_time (timestamp)
- feature_payload (JSONB or normalized columns)
- predicted_class
- predicted_probability
- model_version
- request_id (optional correlation ID)

outcome_labels
- prediction_id (foreign key, unique if one final label)
- observed_label
- label_time
- label_source
```

Use an explicit join on prediction ID, not row order. Track label coverage and label delay. Exclude unlabeled events from accuracy/F1 calculations; report the labeled sample size alongside every performance metric.

7. Step-by-step implementation plan

Milestone 0 — Define the experiment and baseline

- Create a README with problem statement, target definition, dataset source, success metrics and known limitations.
- Record the dataset checksum, feature list, target mapping, split strategy and random seed.
- Train a simple baseline classifier and save baseline metrics. Include at least precision, recall, F1, ROC-AUC where appropriate, confusion matrix and positive-class prevalence.
- Write down a candidate acceptance policy before running the retraining experiment.

Exit condition: a clean training command runs from scratch and produces a versioned model artifact plus a metrics report.

Milestone 1 — Train and package the first model

Implement a training module that loads data, validates columns, splits data, fits preprocessing and the estimator, evaluates it, and serializes the full pipeline. Prefer saving a complete preprocessing-plus-model pipeline to avoid mismatches between training and API inference.

```
# Conceptual training flow; implement as a reusable Python function
load dataset
validate schema and target
split train / validation / test
fit preprocessing + classifier on training data
select/tune using validation data only
evaluate final candidate on held-out test data
save artifact and metrics + feature schema + run metadata
```

- Use a fixed random seed for baseline reproducibility.
- Do not tune on the test set.
- Document how missing values, categorical variables and class imbalance are handled.
- Start with one baseline algorithm; add XGBoost or Optuna only after the end-to-end lifecycle works.

Exit condition: training can be repeated and yields a model that loads in a separate process and produces valid probabilities/predictions.

Milestone 2 — Build the prediction API

Required endpoints

Endpoint	Purpose	Expected behavior
POST /predict	Return churn prediction and probability	Validate schema; log prediction ID, model version and latency
GET /health	Liveness check	Returns service process status
GET /ready	Readiness check	Confirms DB and model are available
GET /metrics	Prometheus scrape target	Numeric metrics; no private customer values
GET /model-info	Expose active model metadata	Returns approved version and load time

- Use Pydantic request/response schemas and clear validation errors.
- Load the model once at startup rather than reloading it for every request.
- Log a prediction event only after validating the input; handle database failure explicitly.
- Include a request/correlation ID, but never use it as a Prometheus metric label.
- Add API tests for valid requests, missing fields, invalid types, database failure and model-loading failure.

Exit condition: the API serves predictions locally and stores a traceable event for each successful prediction.

Milestone 3 — Add PostgreSQL and label delivery

- Create database tables/migrations for prediction events, outcome labels, monitoring runs, retraining runs and model promotion decisions.
- Use SQLAlchemy sessions with bounded transactions and connection settings.
- Write an idempotent label-delivery script that selects eligible prediction IDs and attaches simulated outcomes.
- Prevent duplicate final labels; define how corrections or multiple label updates would be handled.
- Create queries to retrieve recent feature windows, labels and label coverage.

Exit condition: you can predict, inspect a stored event, later attach a label, and compute metrics only for labeled records.

Milestone 4 — Add monitoring and drift detection

- Create a scheduled monitoring job that retrieves a recent production window and compares it with the fixed reference dataset.
- Run data-quality checks for missing columns, unexpected categories, invalid ranges and missing values.
- Run Evidently drift checks on suitable features and prediction outputs.
- Persist the report summary, window boundaries, row count, feature-level results and timestamp.
- Export summary values to Prometheus, such as drift status, share of drifting features, label coverage and last successful monitor time.
- Use a minimum sample threshold before interpreting drift results; small windows can be noisy.

Choose drift thresholds empirically using baseline replays and injected-shift experiments. Avoid treating a default threshold as universally correct.

Exit condition: a baseline window is stable, an injected distribution shift can be detected, and each monitoring run is auditable.

Milestone 5 — Add Prometheus, Grafana and alerting

- Expose counters/histograms for requests, errors and latency, plus bounded gauges/counters for monitoring outcomes.
- Configure Prometheus to scrape the API and monitoring metrics.
- Create Grafana panels for request rate, error rate, latency percentiles, prediction distribution, drift status, label coverage, F1/recall when labels exist, retraining runs and active model version.
- Define alerts for sustained service errors, high latency, missing monitoring runs, drift status and measured performance degradation.
- Route alerts through Alertmanager to a small webhook receiver. Deduplicate by alert fingerprint and incident/window ID.

A drift alert should normally request investigation or evaluate a policy; it should not directly deploy a new model.

Milestone 6 — Build retraining as a manual command first

Before introducing Airflow, implement a single command or Python function that runs the full pipeline deterministically:

- Fetch the latest eligible, labeled training data.
- Validate schema, missingness, ranges, duplicates and minimum sample counts.
- Create a versioned training dataset snapshot.
- Train a candidate model; optionally run a bounded Optuna search.
- Evaluate candidate and champion on the same held-out evaluation set.
- Apply all quality gates and record each gate result.

- Register the candidate in MLflow whether accepted or rejected, with clear tags and metrics.
- Promote only if all required gates pass; otherwise preserve the champion.
- Write a retraining-run record and emit success/failure metrics.

Critical: do not evaluate the candidate on training data and compare that score with the champion's historical test score. Use the same evaluation protocol and a comparable holdout dataset to make a fair comparison.

Milestone 7 — Define champion/challenger quality gates

Example policy — calibrate using experiments, not guesswork

Gate	Example rule	Reason
Data validity	Required schema and validation checks pass	Avoid training on malformed data
Minimum data	Enough newly labeled examples; configured minimum per class	Reduce unstable estimates
Primary metric	Candidate F1 or business-aligned metric exceeds champion by a configured margin, or is non-inferior within a tolerance	Avoid promoting a worse model
Safety metric	Recall or another critical metric does not fall below a defined floor	Protect important cases
Segment checks	No monitored segment crosses an unacceptable degradation threshold	Avoid hiding localized failures
Operational checks	Artifact loads and API smoke test succeeds	Catch packaging failures
Reproducibility	Training run metadata and artifact are stored	Enable audit and rollback

For small datasets, metric differences may be sampling noise. Use confidence intervals or bootstrap comparisons where feasible, and define minimum practical improvements. If no robust improvement is shown, keep the champion. A candidate can also be rejected because evidence is insufficient, not only because its point estimate is lower.

Milestone 8 — Add MLflow registry and controlled model loading

- Track parameters, code revision, dataset version, validation results, evaluation metrics and artifacts for every training run.
- Register model artifacts and identify the active champion with an explicit alias or a small application-controlled pointer, depending on the MLflow version and deployment design.
- Separate candidate registration from promotion. Registration alone must not change production.
- Make model loading atomic: load and smoke-test the candidate before switching the active in-memory model.
- Persist the last-known-good version so the service can recover if the registry is unavailable.
- Record the exact model version used for every prediction.

For a single-process local demo, a safe model refresh can poll for the approved version, load it into a temporary variable, run a smoke test, then swap the active reference. For multi-worker or distributed deployment, use a coordinated deployment mechanism rather than assuming one process updates all workers.

Milestone 9 — Orchestrate with Airflow

Only add Airflow after the manual retraining pipeline is tested. Implement a DAG with explicit task boundaries and persisted outputs.

```
DAG tasks (conceptual)
1. check_trigger_and_cooldown
2. fetch_eligible_labeled_data
3. validate_data
4. create_dataset_snapshot
5. train_candidate
6. evaluate_candidate_vs_champion
```

```
7. apply_quality_gates
8. register_candidate
9. promote_if_approved
10. smoke_test_serving
11. record_result_and_notify
```

- Set retry policies for transient failures, but do not retry data-quality rejection as though it were a network error.
- Use run IDs and idempotent operations to avoid duplicate registrations or promotions.
- Set concurrency limits and a cooldown so repeated alerts do not launch many expensive training jobs.
- Provide a scheduled fallback run, but let policy decide whether training is warranted.
- If the alert webhook cannot contact Airflow, store the event and retry delivery rather than losing it.

Milestone 10 — Docker Compose and CI/CD

- Create a Dockerfile for the API and a Compose stack for API, PostgreSQL, MLflow, Prometheus, Grafana, Alertmanager and Airflow as the project matures.
- Use health checks and `depends_on` health conditions where supported; startup ordering alone does not prove a dependency is ready.
- Use named volumes for database and MLflow artifacts so data survives container restarts.
- Configure service settings through environment variables and provide a `.env.example` with safe placeholders.
- GitHub Actions should run formatting/lint checks, unit tests, integration tests where practical, and a container build.
- For this project, CI should validate software changes. Model promotion should still require the explicit quality-gate workflow.

8. Monitoring, drift detection and delayed labels

8.1 Four monitoring questions

Question	Signals	Interpretation
Is the service healthy?	Latency, request rate, errors, DB health	Operational observability
Did input data change?	Feature drift, missingness, schema violations	Data distribution or quality change
Did model outputs change?	Prediction rate, probability distribution	Could be input shift, model change or pipeline issue
Is the model less accurate?	F1, precision, recall, calibration on labeled examples	Requires observed labels and enough data

8.2 Reference and current windows

Use a fixed reference dataset from the initial training/baseline period. Define the current window explicitly, for example the last N eligible prediction events or a fixed time interval with a minimum row count. Save both window start/end times and row counts. Do not silently change the reference window after each alert; otherwise a persistent shift can become the new baseline without review.

8.3 Label delay and performance

Model performance should be computed on labeled prediction events. Track the number and proportion of events that have labels, the median or distribution of label delay, and the age of the latest label. When labels arrive late, the performance chart should show the observation window and sample count so that users do not confuse stale or incomplete performance estimates with current truth.

8.4 Trigger policy

Recommended initial policy: drift alone creates a warning and begins evidence collection; measured degradation on sufficient labeled examples may trigger retraining; a scheduled workflow can check data availability and drift periodically. A project experiment may deliberately allow drift to trigger a candidate-training run, but it must still require validation and quality gates before promotion.

9. Automated retraining and model promotion

9.1 Retraining state machine

```
IDLE
  ■■ trigger accepted + cooldown satisfied
       ▼
DATA_CHECK
  ■■ invalid / insufficient data ■■■ REJECTED_DATA
  ▼
TRAINING
  ■■ exception / timeout ■■■■■■■■■■■■ FAILED_TRAINING
  ▼
EVALUATION
  ■■ metric gates fail ■■■■■■■■■■■■■■■ REJECTED_QUALITY
  ▼
REGISTERED_CANDIDATE
  ■■ artifact smoke test fails ■■■■■■ REJECTED_ARTIFACT
  ▼
PROMOTION
  ▼
SERVING_SMOKE_TEST
  ■■ deployment/load fails ■■■■■■■■■■ ROLLBACK
  ▼
SUCCEEDED
```

9.2 Required records

- Trigger source, alert ID, start/end time and cooldown decision.
- Training dataset snapshot/version and validation report.
- Candidate run ID, artifact URI, code revision, dependency version and random seed.
- Champion and candidate metrics evaluated on the same test/holdout sample.
- Each gate result, rejection reason and final action.
- Promoted model version, rollout time, smoke-test result and rollback status.

9.3 Failure handling

- If data validation fails, do not train; mark the run rejected and retain the champion.
- If training fails, retry only transient errors within a configured limit; never promote a partial artifact.
- If evaluation is inconclusive, reject or defer promotion rather than guessing.
- If the candidate cannot load or its smoke test fails, keep the champion active.
- If promotion succeeds but serving health degrades, switch back to the last-known-good model and record the rollback.
- If monitoring is unavailable, surface a stale-monitoring alert; do not interpret missing data as healthy data.

10. API, database and deployment behavior

10.1 Suggested database entities

Table	Important fields	Purpose
prediction_events	prediction_id, event_time, features, prediction, probability, model_version, latency_ms	Trace individual inference events
outcome_labels	prediction_id, observed_label, label_time, source	Attach delayed truth
monitoring_runs	run_id, window_start, window_end, sample_count, drift_summary, report_uri, status	Audit monitoring
retraining_runs	run_id, trigger, dataset_version, status, start/end, failure_reason	Audit orchestration
model_evaluations	run_id, model_version, metric_name, value, split_id	Compare champion and challenger
promotion_events	candidate_version, champion_before, decision, gate_results, timestamp	Audit model changes

10.2 Configuration

Use environment variables for database URLs, MLflow tracking URI, model alias/version, monitoring window sizes, drift thresholds, minimum sample sizes, cooldown, training limits and API settings. Validate configuration at startup. Use `.env.example` for documented names and never put real credentials in source control.

10.3 Compose deployment sequence

- Start PostgreSQL and wait for its health check.
- Initialize or migrate the schema.
- Start MLflow tracking/registry services and ensure artifact storage persists.
- Train and register the initial model, then configure the initial champion.
- Start the API and verify `/health`, `/ready`, `/model-info` and `/metrics`.
- Start Prometheus, Grafana and Alertmanager; verify targets are healthy.
- Start monitoring scheduler and Airflow only after their jobs work independently.
- Run the traffic simulator and inspect the end-to-end trace.

11. Testing and failure scenarios

11.1 Test layers

Layer	Examples	Expected outcome
Unit	Feature validation, metrics, quality gates, cooldown logic	Fast, deterministic pass/fail
API integration	POST /predict, invalid payload, DB unavailable	Correct status, no corrupted log
Data tests	Missing column, invalid category, too few labels	Pipeline rejects data before training
Model tests	Artifact load, feature schema, probability range	Model is safe to serve
Workflow tests	Training exception, retry, duplicate alert	No duplicate or partial promotion
End-to-end	Baseline → drift → labels → train → reject/promote	Traceable lifecycle and measurable result
Recovery	Registry unavailable, candidate load fails, API restart	Last-known-good model remains available

11.2 Required negative experiments

- **Worse challenger:** deliberately train or configure a candidate that underperforms; verify it is rejected.
- **Insufficient labels:** make the label count too small; verify performance is marked unknown/insufficient, not zero or healthy.
- **Bad schema:** remove a required feature or introduce an unexpected type; verify validation fails before training.
- **Training crash:** inject an exception; verify the champion remains active and the failure is visible.
- **Promotion/load failure:** make the candidate artifact unavailable or incompatible; verify no broken model replaces the active one.
- **Duplicate alert:** deliver the same alert twice; verify idempotency and cooldown.
- **Monitoring outage:** stop the monitoring job; verify stale-monitoring alerting.
- **Database outage:** test startup and request behavior when PostgreSQL is unavailable.

11.3 What not to do

- Do not report a synthetic model's improved accuracy as a real-world business outcome.
- Do not promote based only on drift score, training score or one favorable metric.
- Do not compute performance on predictions without observed outcomes.
- Do not allow every incoming alert to trigger an unlimited retraining run.
- Do not make production prediction requests wait for a retraining pipeline.

12. Experimental evaluation and metrics

12.1 Experiment protocol

Run the same predefined protocol for each scenario. Save configuration, dataset snapshot, code revision, random seed, model versions, time windows and results. Use multiple repetitions if stochastic training or sampling materially changes outcomes.

- Experiment A — Baseline stability: replay data similar to the reference distribution and verify that the system does not repeatedly trigger false drift alerts.
- Experiment B — Feature drift without known degradation: shift selected feature distributions and confirm drift detection. Once labels arrive, check whether performance actually changed.
- Experiment C — Performance degradation: create a documented shift that changes the feature/target relationship or replay a dataset with changed relationships. Measure the impact only after labels arrive.
- Experiment D — Successful recovery: retrain on suitable newly labeled data; evaluate candidate and champion on the same holdout; verify promotion only if gates pass.
- Experiment E — Rejection: make a candidate fail a quality gate and verify that the champion remains active.
- Experiment F — Pipeline failure: inject data, training, registry or serving failures and confirm safe behavior.
- Experiment G — Operational incident: optionally inject API latency/error spikes and verify operational alerts are distinguished from model drift.

12.2 Metrics to report

Metric	Definition / measurement	Why it matters
Time to detect drift	Time from injection start to first qualifying drift event	Monitoring responsiveness
Time to labeled performance estimate	Time until enough observed labels exist for a valid metric	Makes label delay explicit
Time to recovery	From defined degradation point to approved model serving and passing smoke test	End-to-end automation speed
Performance change	Champion metric before vs during shift, using comparable labeled samples	Quantifies impact
Recovery performance	Approved model's metric on the same evaluation protocol	Measures recovery without test leakage
False alert rate	Unnecessary drift/performance alerts across baseline runs	Alert quality
Correct rejection rate	Fraction of deliberately bad candidates not promoted	Safety of promotion logic
Promotion error count	Number of candidates promoted despite failing a required gate	Should be zero in test suite
Label coverage	Labeled prediction events / eligible prediction events	Shows how complete performance evidence is
API p50/p95 latency	Latency histogram under stated traffic/load	Service behavior
Retraining failure rate	Failed runs / total attempted runs, by reason	Pipeline reliability

12.3 Example result table

Fill this table with actual measurements from your implementation; do not invent results in the final report.

Scenario	Drift detected?	Labeled performance	Decision	Recovery / outcome
Baseline	Measured	Measured when labels arrive	No retraining unless policy says so	Baseline reference
Feature shift only	Measured	Could remain stable or degrade	Investigate / policy-based	Report actual outcome
Degradation	Measured if detectable	Metric falls on labeled events	Start candidate workflow	Measure elapsed time
Bad candidate	Not applicable	Candidate fails gate	Reject	Champion unchanged
Good candidate	Not applicable	Candidate passes gates	Promote	Verify model version changes
Training failure	Not applicable	Unavailable	Fail safely	Champion unchanged

12.4 Performance metric caveats

For churn, accuracy alone can hide poor detection of churners, especially if the classes are imbalanced. Include precision, recall, F1 and a confusion matrix; consider PR-AUC, ROC-AUC, calibration and business-specific costs when appropriate. Choose the primary metric before looking at candidate results. Keep test data separate from tuning and promotion decisions where possible; if a repeated promotion process uses one holdout many times, create a rolling or fresh evaluation protocol to reduce overfitting to that holdout.

13. Security, reliability and reproducibility

13.1 Security baseline

- Use fake or public data only; do not store real customer information in a portfolio demo.
- Keep credentials in environment variables or local secret storage, not Git.
- Restrict dashboard, MLflow and database ports to local development unless remote access is intentionally secured.
- Validate API inputs and cap request sizes.
- Avoid logging full sensitive payloads. If full features are required for the experiment, document this choice and use non-sensitive data.
- Keep dependencies updated and pinned; scan or review dependency changes.

13.2 Reliability baseline

- Use database migrations and backups for important experiment artifacts.
- Use timeouts, bounded retries, concurrency limits and cooldowns.
- Keep the last-known-good model artifact available locally or in persistent storage.
- Expose the last successful monitoring time and last successful retraining time.
- Make promotion idempotent and record every state transition.
- If the system cannot determine whether a candidate is safe, it should not promote it.

13.3 Reproducibility metadata

- Code commit hash, Python and library versions, dataset checksum, feature schema and split strategy.
- Random seeds, model hyperparameters, preprocessing pipeline and training time.
- Evaluation metrics, sample counts, evaluation window and confidence intervals where feasible.
- MLflow run ID, artifact URI, model version and promotion decision.
- Docker image tag and configuration used in the experiment.

14. Final deliverables and acceptance checklist

14.1 Expected deliverables

- Source code with clear module boundaries and setup instructions.
- README with architecture, commands, endpoint examples, troubleshooting and limitations.
- Docker Compose configuration and environment template.
- Database schema/migrations and sample non-sensitive data.
- Baseline model, training metadata and evaluation report.
- FastAPI endpoints and prediction logging.
- Evidently monitoring reports and stored monitoring-run history.
- Prometheus configuration, alert rules and Grafana dashboards.
- MLflow experiments, registered candidate models and promotion history.
- Airflow DAG or a documented alternative orchestrator.
- Unit, integration, negative-path and end-to-end tests.
- Experiment report with measured results and plots/screenshots.

- A short demo script showing baseline, drift, delayed labels, candidate rejection, candidate promotion and failure handling.

14.2 Definition of done

- A fresh setup can be completed using the documented steps.
- The baseline API serves predictions and stores traceable events.
- The simulator can reproduce documented baseline and drift scenarios.
- The system distinguishes drift alerts from measured performance degradation.
- Performance metrics use observed labels and show sample counts.
- A retraining run records dataset version, model metrics, gate results and final decision.
- A deliberately worse candidate is rejected and the champion stays active.
- A passing candidate can be promoted, loaded and smoke-tested.
- Training, registry or candidate-load failure does not replace the champion.
- Prometheus/Grafana expose service health, drift, labels, model performance and retraining status.
- CI runs tests and lint checks successfully.
- The final report includes actual measurements and explains limitations.

15. Instructions for another AI coding assistant

The following is a reusable implementation brief. Give it to a coding assistant along with this document and ask it to work milestone by milestone.

```
You are implementing the project specified in this document:
“Self-Maintaining ML System: Production Monitoring, Drift Detection and Automated Retraining.”

Rules:
1. Treat this document as the source of truth for scope and behavior.
2. Do not generate the whole repository at once. Implement one milestone at a time.
3. Before coding a milestone, summarize the files to create/change and the design.
4. Write complete, runnable code—not pseudocode—when implementing a milestone.
5. Include tests for normal behavior and failure paths.
6. After each milestone, provide exact commands to run and expected outputs.
7. Do not claim tests passed unless they were actually run.
8. Keep secrets and real customer data out of source control.
9. Pin compatible dependencies and document version assumptions.
10. Prefer a small, reliable core over optional complexity.
11. Never promote a model based on drift alone or training accuracy.
12. Compute model performance only when ground-truth labels are available.
13. Compare champion and challenger under the same evaluation protocol.
14. Reject candidates when evidence is insufficient or any required quality gate fails.
15. Preserve the last-known-good champion on every failure path.
16. Make alerts and promotion operations idempotent, with cooldowns and retry limits.
17. Keep a clear record of dataset version, code revision, metrics, gate decisions and model version.
18. Use public/non-sensitive data and explicitly label simulated experiments.
19. At the end of each milestone, stop and wait for the developer to verify it before proceeding.

Start with Milestone 0: project skeleton, environment, dataset documentation,
baseline training, and tests. Do not start Airflow, Grafana or Kubernetes yet.
```

Suggested build order at a glance

Order	Build	Do not continue until
0	Repo, environment, dataset audit and baseline	Training is repeatable and metrics are saved
1	Preprocessing pipeline and model artifact	Artifact loads in a clean process
2	FastAPI prediction service	Prediction and logging tests pass
3	PostgreSQL labels and history	A label can be attached by prediction ID
4	Monitoring/Evidently job	Baseline and injected drift runs are auditable
5	Prometheus/Grafana/alerts	Dashboards and alerts reflect actual values
6	Manual retraining + quality gates	Bad candidate is rejected in tests
7	MLflow registry and safe model loading	Approved model switch and rollback work
8	Airflow orchestration	Retries, cooldown and idempotency tests pass
9	Docker Compose and CI polish	Fresh startup and end-to-end demo succeed
10	Final experiments and report	All metrics are measured and caveats documented

Appendix A — Practical implementation notes

A.1 Start smaller than the final architecture

The most reliable path is to first run a local Python training script, then a local API, then PostgreSQL, then monitoring, and finally orchestration. Docker Compose should bring the components together, but it should not hide the need to understand each component independently.

A.2 Keep model and API contracts stable

Define one canonical feature schema shared by training, validation, simulator and API. Version the schema if features change. A model that expects a different column order or categorical encoding must not be loaded silently.

A.3 Make the demo easy to review

Provide a single documented experiment command or script that orchestrates a repeatable demo, but keep individual scripts available for debugging. The demo should print or link to the model version, monitoring run ID, retraining run ID, gate results and active champion.

A.4 Explain the limits

A local simulated system demonstrates engineering design and controlled failure behavior. It does not prove that the model would improve a real company's churn outcomes. State which shifts are synthetic, which labels are simulated, how data was split, and whether the evaluation represents a real temporal deployment.

A.5 Suggested final demo narrative

- Show the active champion version and its baseline metrics.
- Send baseline traffic and show stable service/monitoring panels.
- Inject a documented distribution shift and show the drift report.
- Explain that drift alone does not prove accuracy degradation.
- Deliver delayed labels and show performance metrics with labeled sample count.
- Run retraining and show data validation, candidate evaluation and quality gates.
- First demonstrate a candidate rejection and unchanged champion.

- Then demonstrate a candidate passing the gates, promotion and serving smoke test.
- Finally inject a failure and show that the system preserves or restores the last-known-good model.

End of document.